

Modules and Object Orientation

Modules

- Issue to build the big software:
 - How to divide the effort among programmers so that the work can proceed on multiple fronts simultaneously?
- Depends on the concept of information hiding
 - Keep objects and algorithms invisible, to portions of the system that do not need them.
- Information hiding reduces the risk of name conflicts:
 - Less chance that a new name will be same as the one already in use since there are fewer visible names.
- Integrity of data abstractions:
 - Any attempt to access objects outside of the subroutine(s) to which they belong will cause the compiler to issue an “undefined symbol” error message.
- It helps to compartmentalize run-time errors:
 - If a variable takes on an unexpected value, it is generally be the case that the code that modified it is in the variable’s scope.

Encapsulating Data and Subroutines

- Information hiding for nested subroutines is restricted to objects whose lifetime is the same as that of the subroutine in which they are hidden.
- When control returns from a subroutine, its local variables will no longer be available (their values will be omitted).
- Partial solution
 - The static variables of C.
 - The abstraction using static variables is restricted to only one subroutine.
 - Does not work with the abstractions whose interface needs to consist of more than one subroutine

Modules

- Our aim is to build a stack abstraction.
 - We want to hide the internal structure of the stack from the rest of the program, such that it can be accessed through its push and pop operations.
 - It can be done using ***module*** in many languages.
- Modules
- It is a collection of objects (subroutines, variables, types, etc.) to be encapsulated
 - Objects inside the module are visible to each other.
 - Objects on the inside are not visible on the outside unless explicitly exported.
 - Objects outside are not visible on the inside unless explicitly imported.

Modules examples

- Clu as clusters, Modula (1, 2, and 3), Turing and Ada 83.
- Haskell, C++, Java, and C# and in major scripting languages.
- Ada, Java, and Perl, use the term package instead of module.
- C++, C#, and PHP, use namespace.
- In C, modules can be emulated to some degree through use of the separate compilation facilities.

Example:Stack

```
// interface
namespace Stack{
    void push(char);
    char pop();
}

void f()
{
    Stack::push('c');
    if (Stack::pop() != 'c')
        error("impossible");
}
```

```
// implementation
namespace Stack{
    const int max_size = 200;
    char v[max_size];
    int top = 0;

    void push(char c) {
        /* check for overflow and push c
        */ }

    char pop() { /* check for
    underflow and pop */ }
}
```

Example:Stack

- The user code is insulated from the data representation of Stack by the code implementing `Stack::push()` and `Stack::pop()`.
- The user doesn't have to concern about the following two things:
 - Implementation of Stack: whether the stack is implemented using an array, or any other means.
 - The implementation can be changed without affecting user code.
- The above (previous slide) representation of Stack module is one way of representation.
- Other representations can be with separate compilation of files.

Multiple Stacks

- Modules enable us with
 - Encapsulation
 - Information Hiding
- Disadvantages
 - Does not provide data abstraction
 - What If we want to use two stacks simultaneously, we have to replicate that module again.
 - Approach of modularity does not help us create multiple instances at run time.

Class

- Class supports data abstraction/ abstract data type
- Getter and setter methods to get and set values
- Constructors and destructors are provided
- Implicit call to constructor (provided by language implementation) when an object is created and destructor when the object goes out of scope.
- All this is possible with an object-orientation.

Objects

- Runtime representation for a Class
- Identity
 - Every object has got its own identity.
- State
 - State tells us about the type or the value of that object
- Behavior
 - Behaviour tells us about the operations that the object can perform.

Object Orientation

- We need to address following questions before implementing OOP concepts.
- How is the access control supported?
- How is the function represented at runtime?
- How are objects represented at runtime?
- How to make function calls to these objects?

Object Orientation Cont..

- To support access control
 - The access specifiers can be stored in Type Descriptor Table (TDT)/Symbol Table.
 - The scope can be known at compile time using TDT/Symbol Table.
 - Before each access, scope can be checked and then only access can be granted (public or private).
 - For an attempt to access a member inside the class or member function, we check if it is public/private, from the TDT and flag an error if there is a violation.
 - Once the emitted code is generated, we discard the TDT/Symbol Table as this (access specifier) information is not needed at runtime (Access specifiers are not needed at run time)

Object Orientation Cont..

- Function representation
 - We would have a single copy of all the functions created.
 - All the objects share the code segment that has function definitions of all the functions defined.
- How to make function calls to these objects?
- At point of call, a pointer to the object calling function is passed as a hidden parameter.
- This points to the current object corresponding to which this function is called.
- Example:
 - `S1.push('x');` // Push passes 'x' as well as a hidden pointer to object S1.
 - It is called as “this” pointer.

Object Orientation Cont..

- How is an object represented at runtime?
- At runtime, an object looks similar to struct only, the data members are kept for an object.
- If an object of another class is a data member of a class, then a pointer to that object of another class can be stored.
- Concept is similar to struct, so its implementation is also similar, i.e., padding bytes, etc.

Initialization and Finalization

- Most object-oriented languages have mechanisms to initialize an object automatically at the beginning of its lifetime.
- Written in the form of a subroutine - constructor.
- Constructor:
 - Does not allocate space; it initializes space that has already been allocated.
- Destructor:
 - Mechanism to finalize an object automatically at the end of its lifetime.

Issues related to Initialization and Finalization

- Choosing a constructor:
- A class may have zero, one, or many distinct constructors.
- Different constructors may have different names
- Need to distinguish the constructors based on number and types of arguments.
- Ex., C++, Java, and C#
 - The constructors are like overloaded subroutines
 - They must be distinguished by their numbers
- Smalltalk and Eiffel
 - Different constructors can have different names;
 - Code that creates an object must name a constructor explicitly

Issues related to Initialization and Finalization Cont..

- References and values:
- References:
 - If variables are references, then every object must be created explicitly.
 - Simula, Smalltalk, Python, Ruby, and Java, use a programming model in which variables refer to objects.
- Variables:
 - If variables are values, then object creation can happen implicitly as a result of elaboration.
 - The language must allow objects to begin their lifetime uninitialized.
 - C++, Modula-3, Ada 95, and Oberon allow a variable to have a value that is an object.

Example Constructors

- C++ variable of class type foo is declared with no initial value, then the Compiler will call foo's zero-argument constructor.

```
foo b;    // calls foo::foo()
```

```
foo b(10, 'x');    // // calls foo::foo(int, char)
```

Copy constructor

- Declare a new object whose initial value is “the same” as that of the existing object.
- Initialization
 - `foo a; // calls foo::foo()`
 - `foo c = a; // calls foo::foo(foo&)`
- Assignment
 - `foo a, c, d; // calls foo::foo() three times`
 - `c = a; // calls foo::operator=(foo&)`

Issues related to Initialization and Finalization Cont..

- Execution order
- In C++:
- For the created object (Y1) of a derived class(Y),
 - The constructors for any base classes (X, Y, Z) will be executed, (outermost first, say X) before the constructor for the derived class (Y).
- Objects of some class (say Y1 is an object of class Y) are members of a Class (X)
 - The constructors for the members will be called before the constructor for the object in which they are contained.

Issues related to Initialization and Finalization Cont..

- Garbage collection:
- Most object-oriented languages provide some sort of destructor mechanism.
- Purpose
- To facilitate manual storage reclamation in languages like C++.
- For the language implementation that collects garbage automatically, no need to have destructors.

References

- Programming Language Pragmatics by Michael L. Scott, 3rd Edition (Elsevier)